

MATLAB Example: Sorting Students into Groups

Purpose of This Example

In this example, we use MATLAB to determine whether a class of students can be divided evenly into groups of **2, 3, or 4**.

The number of students is randomly generated between **70 and 130**. Then, MATLAB calculates how many students would remain if we tried to divide the class into groups of 2, 3, or 4.

This example helps you learn how to:

- Generate random integers using `randi`
- Use the remainder function `rem`
- Print formatted output using `fprintf`
- Continue long text statements across multiple lines using `...`

MATLAB Code

```
% Author: Prof. Lund
% Purpose: To sort students into groups

% Generate the number of students in the class
numStudents = randi([70 130]);
fprintf('There are %d students.\n', numStudents);

% Arrange them into groups
groupsOf2 = rem(numStudents, 2);
groupsOf3 = rem(numStudents, 3);
groupsOf4 = rem(numStudents, 4);

% Report how many students remain
fprintf(['We can make groups of 2 with ' ...
        '%d students remaining.\n'], groupsOf2);

fprintf(['We can make groups of 3 with ' ...
        '%d students remaining.\n'], groupsOf3);

fprintf(['We can make groups of 4 with ' ...
        '%d students remaining.\n'], groupsOf4);
```

Detailed Explanation

1. Generating the Number of Students

```
numStudents = randi([70 130]);
```

The command `randi([70 130])` generates a random integer between **70 and 130**.

For example, MATLAB may generate:

```
numStudents = 96
```

or:

```
numStudents = 117
```

Each time you run the code, MATLAB may generate a different number of students.

2. Printing the Number of Students

```
fprintf('There are %d students.\n', numStudents);
```

The function `fprintf` is used to print text in the Command Window.

Here, `%d` is a placeholder for an integer number.

So, if:

```
numStudents = 96
```

then MATLAB prints:

```
There are 96 students.
```

The symbol `\n` means “go to a new line” after printing the sentence.

3. Finding the Remainder

The next part of the code checks whether the students can be divided evenly into groups of 2, 3, or 4.

```
groupsOf2 = rem(numStudents, 2);  
groupsOf3 = rem(numStudents, 3);  
groupsOf4 = rem(numStudents, 4);
```

The MATLAB function `rem(a,b)` calculates the remainder after dividing `a` by `b`.

For example:

```
rem(10, 2)
```

gives:

0

because 10 can be divided evenly by 2.

Another example:

```
rem(10, 3)
```

gives:

1

because 10 divided by 3 gives 3 full groups, with 1 student remaining.

4. Meaning of Each Line

Groups of 2

```
groupsOf2 = rem(numStudents, 2);
```

This line calculates how many students remain if the class is divided into groups of 2.

If the result is 0, then all students can be placed into groups of 2.

Example:

```
rem(96, 2)
```

Result:

0

This means 96 students can be divided evenly into groups of 2.

Groups of 3

```
groupsOf3 = rem(numStudents, 3);
```

This line calculates how many students remain if the class is divided into groups of 3.

Example:

```
rem(96, 3)
```

Result:

0

This means 96 students can be divided evenly into groups of 3.

Another example:

```
rem(97, 3)
```

Result:

1

This means if we divide 97 students into groups of 3, one student will remain.

Groups of 4

```
groupsOf4 = rem(numStudents, 4);
```

This line calculates how many students remain if the class is divided into groups of 4.

Example:

```
rem(96, 4)
```

Result:

0

This means 96 students can be divided evenly into groups of 4.

5. Printing the Results

```
fprintf(['We can make groups of 2 with ' ...  
        '%d students remaining.\n'], groupsOf2);
```

This command prints the number of students remaining when making groups of 2.

For example, if:

```
groupsOf2 = 0
```

then MATLAB prints:

```
We can make groups of 2 with 0 students remaining.
```

The same idea is used for groups of 3 and 4.

6. Using the Continuation Operator ...

In this code, we see:

```
fprintf(['We can make groups of 2 with ' ...  
        '%d students remaining.\n'], groupsOf2);
```

The symbol:

```
...
```

is called the continuation operator.

It tells MATLAB that the command continues on the next line.

This is useful when a line of code is too long.

For long strings inside `fprintf`, we also use square brackets:

```
[ ... ]
```

This combines the text into one complete string.

So this:

```
fprintf(['We can make groups of 2 with ' ...  
        '%d students remaining.\n'], groupsOf2);
```

is equivalent to writing:

```
fprintf('We can make groups of 2 with %d students remaining.\n', groupsOf2);
```

Both versions give the same result.

Example Output

Because the number of students is random, your output may be different each time.

For example, MATLAB may generate:

```
numStudents = 97
```

Then the output would be:

```
There are 97 students.  
We can make groups of 2 with 1 students remaining.  
We can make groups of 3 with 1 students remaining.  
We can make groups of 4 with 1 students remaining.
```

This means:

- 97 students cannot be divided evenly into groups of 2.
- 97 students cannot be divided evenly into groups of 3.
- 97 students cannot be divided evenly into groups of 4.

There will be 1 student remaining in each case.

Important Note About Variable Names

In this example, the variables are named:

```
groupsOf2  
groupsOf3  
groupsOf4
```

However, these variables do **not** store the number of groups.

They store the number of students remaining after division.

A clearer name could be:

```
remainingForGroupsOf2  
remainingForGroupsOf3  
remainingForGroupsOf4
```

For example:

```
remainingForGroupsOf2 = rem(numStudents, 2);
```

This name is longer, but it explains the meaning more clearly.

Improved Version for Students

```
% Purpose: To check whether students can be divided evenly into groups  
  
clc  
clear  
  
% Generate a random number of students between 70 and 130  
numStudents = randi([70 130]);  
  
% Print the number of students  
fprintf('There are %d students.\n', numStudents);  
  
% Calculate the number of students remaining for each group size  
remainingForGroupsOf2 = rem(numStudents, 2);  
remainingForGroupsOf3 = rem(numStudents, 3);  
remainingForGroupsOf4 = rem(numStudents, 4);  
  
% Print the results  
fprintf('Groups of 2: %d student(s) remaining.\n', remainingForGroupsOf2);  
fprintf('Groups of 3: %d student(s) remaining.\n', remainingForGroupsOf3);  
fprintf('Groups of 4: %d student(s) remaining.\n', remainingForGroupsOf4);
```

Practice Question for Students

Professor Ruffell is teaching a course next term. The course may have between **70 and 130 students**.

She wants students to work on a group project. The groups can have **2, 3, or 4** students.

Write a MATLAB code that:

1. Randomly generates the number of students.
 2. Checks how many students remain if groups of 2 are formed.
 3. Checks how many students remain if groups of 3 are formed.
 4. Checks how many students remain if groups of 4 are formed.
 5. Prints the results clearly in the Command Window.
-

Practice Answer

```
clc
clear

numStudents = randi([70 130]);

fprintf('There are %d students in the class.\n', numStudents);

remaining2 = rem(numStudents, 2);
remaining3 = rem(numStudents, 3);
remaining4 = rem(numStudents, 4);

fprintf('If groups of 2 are used, %d student(s) remain.\n', remaining2);
fprintf('If groups of 3 are used, %d student(s) remain.\n', remaining3);
fprintf('If groups of 4 are used, %d student(s) remain.\n', remaining4);
```

Extra Challenge

Modify the code to also calculate the number of full groups.

For example, if there are 97 students and groups of 4 are used:

```
floor(97/4)
```

gives:

```
24
```

This means we can make 24 full groups of 4, with 1 student remaining.

Example code:

```
numGroupsOf4 = floor(numStudents/4);
remainingForGroupsOf4 = rem(numStudents, 4);

fprintf('We can make %d full groups of 4.\n', numGroupsOf4);
fprintf('%d student(s) will remain.\n', remainingForGroupsOf4);
```

Key MATLAB Commands Used

Command	Meaning
<code>randi([70 130])</code>	Generates a random integer between 70 and 130
<code>rem(a,b)</code>	Finds the remainder after dividing <code>a</code> by <code>b</code>
<code>fprintf</code>	Prints formatted text in the Command Window
<code>%d</code>	Placeholder for an integer
<code>\n</code>	Starts a new line
<code>...</code>	Continues a command on the next line
<code>floor(x)</code>	Rounds a number down to the nearest integer